

SIMATS ENGINEERING
Slot: C, Course Code: ITA0501, Course Name: Computer Vision

Duration: 60 mins

Marks: 20

Activity Tool 1: Experiments (High)

Assessment Tool Weightage: 40%

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Q. No.	Questions
1	Preprocessing Impact Analysis Design and perform an experiment to study the impact of preprocessing techniques such as noise removal and normalization on image quality and feature detection. Explain the theoretical concepts behind preprocessing methods and justify the selection of techniques used. Implement the preprocessing process using suitable software/tools, document the workflow and outputs systematically, and analyze how preprocessing influences feature detection accuracy and reliability.
2	Image Representation Study Conduct an experiment to convert a given image into grayscale and binary representations. Explain the significance of different image representations in computer vision applications. Design the conversion procedure using appropriate tools, document the generated outputs clearly, and analyze how image representation affects edge detection and feature extraction performance.
3	Edge Detection Comparison Design an experiment to compare Sobel and Canny edge detection techniques on the same input image. Explain the underlying concepts and operational principles of both methods. Apply the techniques using suitable tools/software, document the detected edges systematically, and analyze the comparative effectiveness of each method for different image conditions and applications.
4	Harris Corner Detection Analysis Perform Harris corner detection on images containing varying textures and patterns. Explain the concept of corner detection and its importance in computer vision tasks. Use appropriate parameters and tools to execute the experiment, document the detected corner points clearly, and analyze the distribution, accuracy, and significance of the detected corners.
5	Effect of Noise on Corner Detection Design an experiment to study the effect of noise on Harris corner detection performance. Explain the purpose of introducing noise and its influence on image features. Add noise systematically, perform corner detection using suitable tools, document observations for original and noisy images, and analyze the impact of noise along with possible improvement techniques.

Code of Conduct:

I SOMISETTY VENKATA SAI NAVEEN KUMAR (192425176) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student
SOMISETTY VENKATA SAI NAVEEN KUMAR
192425176

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

Signature of the Course Faculty

Experiment 1: Preprocessing Impact Analysis

Aim

To study the impact of preprocessing techniques such as noise removal and normalization on image quality and feature detection performance.

Objective

- To understand the importance of image preprocessing.
- To remove unwanted noise from an image.
- To normalize image intensity values.
- To compare feature detection before and after preprocessing.
- To analyze how preprocessing improves feature detection reliability.

Theory

Image preprocessing is the initial stage of a computer vision system. Images captured from cameras may contain noise, uneven illumination, blur, and intensity variations. These unwanted effects can reduce the accuracy of feature detection algorithms.

Noise Removal

Noise is unwanted variation in pixel intensity. Common noise-removal techniques include:

- **Gaussian Blur:** Reduces Gaussian noise using a weighted averaging operation.
- **Median Filter:** Replaces each pixel with the median value of its neighborhood and is effective against salt-and-pepper noise.
- **Bilateral Filter:** Smooths an image while preserving important edges.

Normalization

Normalization adjusts pixel intensity values to a standard range. It improves consistency when images have different brightness or contrast levels.

For an image with minimum intensity ($I_{\min}$) and maximum intensity ($I_{\max}$), normalization can be expressed as:

$$I_{\text{norm}} = \frac{I - I_{\min}}{I_{\max} - I_{\min}}$$

The resulting values are commonly scaled to the range 0–255.

Software and Tools

- Python
- OpenCV
- NumPy

- Google Colab or Jupyter Notebook

Procedure

1. Load the input image.
2. Convert the image to grayscale.
3. Add or use an image containing noise.
4. Detect features from the original/noisy image.
5. Apply Gaussian filtering for noise removal.
6. Apply intensity normalization.
7. Detect features again.
8. Compare the detected features before and after preprocessing.
9. Analyze the improvement.

Python Program

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

# Load image
img = cv2.imread("input.jpg")

if img is None:
    raise FileNotFoundError("input.jpg not found")

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Add Gaussian noise
noise = np.random.normal(0, 25, gray.shape)
noisy = gray.astype(np.float32) + noise
noisy = np.clip(noisy, 0, 255).astype(np.uint8)
```

```
# Noise removal
denoised = cv2.GaussianBlur(noisy, (5, 5), 0)

# Normalization
normalized = cv2.normalize(
    denoised, None, 0, 255, cv2.NORM_MINMAX
)

# Harris corner detection function
def detect_corners(image):
    corners = cv2.cornerHarris(
        np.float32(image), 2, 3, 0.04
    )
    corners = cv2.dilate(corners, None)

    result = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)
    result[corners > 0.01 * corners.max()] = [0, 0, 255]

    return result, np.sum(corners > 0.01 * corners.max())

# Detect corners
original_result, original_count = detect_corners(gray)
noisy_result, noisy_count = detect_corners(noisy)
processed_result, processed_count = detect_corners(normalized)

# Display results
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(gray, cmap="gray")
```

```
plt.title("Original")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 2)
```

```
plt.imshow(noisy, cmap="gray")
```

```
plt.title("Noisy")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 3)
```

```
plt.imshow(denoised, cmap="gray")
```

```
plt.title("Denoised")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 4)
```

```
plt.imshow(normalized, cmap="gray")
```

```
plt.title("Normalized")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 5)
```

```
plt.imshow(noisy_result)
```

```
plt.title("Corners - Noisy")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 6)
```

```
plt.imshow(processed_result)
```

```
plt.title("Corners - Preprocessed")
```

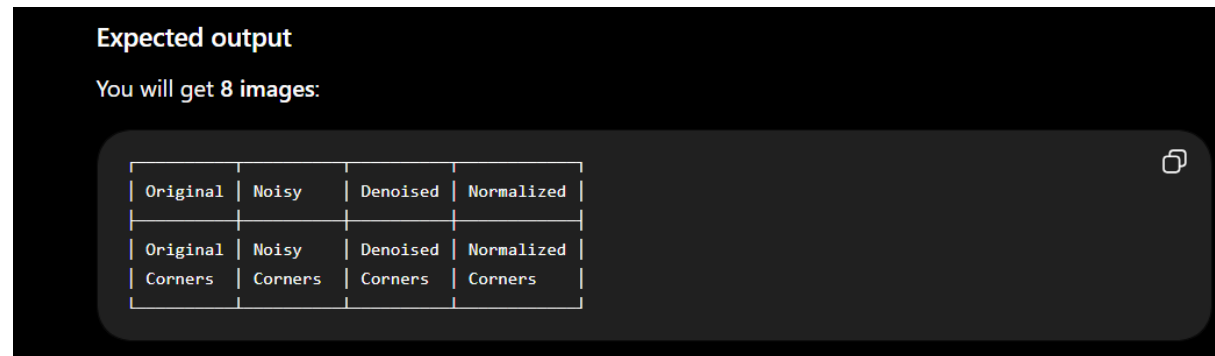
```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print("Original corner response:", original_count)
print("Noisy corner response:", noisy_count)
print("Preprocessed corner response:", processed_count)
```

Expected Output



Observation

The noisy image generally produces additional unwanted feature responses. After Gaussian filtering and normalization, random variations are reduced and meaningful structures become more prominent.

Analysis

Preprocessing improves the quality and consistency of feature detection. Noise removal prevents false feature responses, while normalization makes intensity variations more consistent. Therefore, preprocessing should normally be performed before feature detection when the input images contain significant noise or illumination variations.

Result

The experiment demonstrates that noise removal and normalization improve image quality and generally make feature detection more reliable and accurate.

Experiment 2: Image Representation Study

Aim

To convert an input image into grayscale and binary representations and study their effects on edge detection and feature extraction.

Objective

- To understand different image representations.

- To convert a color image into grayscale.
- To convert a grayscale image into binary form.
- To study the effect of representation on edge detection.
- To understand the importance of image representation in computer vision.

Theory

A digital image can be represented in different forms depending on the application.

Color Image

A color image normally contains three channels:

- Red
- Green
- Blue

Therefore, a color image contains more information but requires greater computational resources.

Grayscale Image

A grayscale image contains only intensity information. Each pixel generally has a value from 0 to 255.

The conversion can be approximated by:

$$\begin{bmatrix} \text{Gray} = 0.299R + 0.587G + 0.114B \end{bmatrix}$$

Grayscale images are widely used for edge detection, corner detection, segmentation, and feature extraction.

Binary Image

A binary image contains only two intensity values, normally 0 and 255.

Thresholding is used to convert a grayscale image into a binary image:

$$\begin{bmatrix} B(x,y) = \begin{cases} 255 & \text{if } I(x,y) > T \\ 0 & \text{if } I(x,y) \leq T \end{cases} \end{bmatrix}$$

where (T) is the threshold value.

Software and Tools

- Python
- OpenCV
- NumPy
- Matplotlib
- Google Colab/Jupyter Notebook

Procedure

1. Load a color image.
2. Display the original image.
3. Convert the image into grayscale.
4. Apply thresholding to generate a binary image.
5. Apply edge detection to the grayscale image.
6. Apply edge detection to the binary image.
7. Compare the generated outputs.
8. Analyze the effect of image representation.

Python Program

```
import cv2
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("input.jpg")

if img is None:
    raise FileNotFoundError("input.jpg not found")

# Convert BGR to RGB for displaying
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```



```
# Binary using threshold
_, binary = cv2.threshold(
    gray, 127, 255, cv2.THRESH_BINARY
)

# Edge detection
gray_edges = cv2.Canny(gray, 100, 200)
binary_edges = cv2.Canny(binary, 100, 200)

# Display
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(rgb)
plt.title("Original Color")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(binary, cmap="gray")
plt.title("Binary")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(gray_edges, cmap="gray")
plt.title("Edges from Grayscale")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 5)
```

```
plt.imshow(binary_edges, cmap="gray")
```

```
plt.title("Edges from Binary")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

Expected Output

Expected output

You will see:

Original → Grayscale → Binary → Grayscale Edges → Binary Edges

Observation

The grayscale image retains intensity information and provides smoother transitions between objects. The binary representation simplifies the image into foreground and background regions.

Grayscale images are generally more suitable for detailed edge and feature analysis, while binary images are useful for object segmentation, shape analysis, and contour extraction.

Analysis

Image representation directly affects computer vision performance. Color images contain maximum information but require more computation. Grayscale reduces computational complexity while retaining useful structural information. Binary representation provides strong separation between foreground and background but may lose fine intensity details.

Result

The experiment successfully converts an image into grayscale and binary representations. The grayscale representation provides useful information for edge and feature detection, whereas binary representation is useful for segmentation and shape-based analysis.

Experiment 3: Edge Detection Comparison Using Sobel and Canny

Aim

To compare Sobel and Canny edge detection techniques using the same input image and analyze their effectiveness under different image conditions.

Objective

- To understand the concept of edge detection.
- To study the Sobel operator.
- To study the Canny edge detector.
- To compare the detected edges.
- To determine the advantages and limitations of each technique.

Theory

An edge represents a significant change in image intensity. Edge detection is an important preprocessing step in object detection, segmentation, shape analysis, and feature extraction.

Sobel Edge Detection

The Sobel operator calculates image intensity gradients in horizontal and vertical directions.

The horizontal Sobel kernel is:

```
[  
G_x=  
\begin{bmatrix}  
-1&0&1\  
-2&0&2\  
-1&0&1  
\end{bmatrix}  
]
```

The vertical Sobel kernel is:

```
[  
G_y=  
\begin{bmatrix}  
-1&-2&-1\  
0&0&0\  
1&2&1  
\end{bmatrix}  
]
```

The gradient magnitude can be calculated as:

```
[  
G=\sqrt{G_x^2+G_y^2}  
]
```

Sobel is simple and computationally efficient but may detect unwanted noise.

Canny Edge Detection

Canny is a multi-stage edge detection algorithm.

Its main stages are:

1. Gaussian noise reduction.
2. Gradient calculation.
3. Non-maximum suppression.
4. Double thresholding.
5. Hysteresis edge tracking.

Canny generally produces thin and well-connected edges.

Procedure

1. Load the same input image.
2. Convert it to grayscale.
3. Apply Sobel edge detection.
4. Apply Canny edge detection.
5. Display both results.
6. Compare edge thickness, continuity, noise sensitivity, and computational complexity.

Python Program

```
import cv2
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("input.jpg")

if img is None:
    raise FileNotFoundError("input.jpg not found")

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Sobel
sobel_x = cv2.Sobel(
    gray, cv2.CV_64F, 1, 0, ksize=3
```

```
)
```

```
sobel_y = cv2.Sobel(  
    gray, cv2.CV_64F, 0, 1, ksize=3  
)
```

```
sobel = cv2.magnitude(sobel_x, sobel_y)
```

```
# Convert to uint8
```

```
sobel = cv2.convertScaleAbs(sobel)
```

```
# Canny
```

```
canny = cv2.Canny(  
    gray, 100, 200  
)
```

```
# Display
```

```
plt.figure(figsize=(12, 6))
```

```
plt.subplot(1, 3, 1)
```

```
plt.imshow(gray, cmap="gray")
```

```
plt.title("Original Grayscale")
```

```
plt.axis("off")
```

```
plt.subplot(1, 3, 2)
```

```
plt.imshow(sobel, cmap="gray")
```

```
plt.title("Sobel Edges")
```

```
plt.axis("off")
```

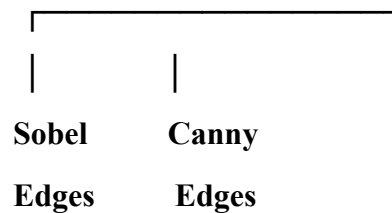
```
plt.subplot(1, 3, 3)
```

```
plt.imshow(canny, cmap="gray")  
plt.title("Canny Edges")  
plt.axis("off")
```

```
plt.tight_layout()  
plt.show()
```

Expected Output

Original Grayscale



Comparison

Feature	Sobel	Canny
Principle	Gradient calculation	Multi-stage edge detection
Noise handling	Moderate	Better
Edge thickness	Relatively thick	Thin
Edge continuity	Moderate	Generally high
Parameters	Kernel size	Two thresholds + processing stages
Computational complexity	Lower	Higher
Accuracy	Moderate	Generally higher

Observation

Sobel detects intensity changes effectively and is computationally simple. However, it may produce thicker edges and unwanted responses in noisy regions.

Canny generally produces thinner and better-connected edges because it includes smoothing, non-maximum suppression, and hysteresis.

Analysis

For simple and fast applications, Sobel can be appropriate. Canny is preferable when accurate and well-defined edges are required. In noisy images, Canny usually performs better because Gaussian smoothing is included in its processing pipeline.

Result

The experiment successfully compares Sobel and Canny edge detection. Canny generally provides cleaner, thinner, and more continuous edges, while Sobel provides a simpler and faster gradient-based solution.

Experiment 4: Harris Corner Detection Analysis

Aim

To perform Harris corner detection on images containing different textures and patterns and analyze the distribution and significance of detected corners.

Objective

- To understand corner detection.
- To study the Harris corner detection algorithm.
- To detect corners in different image patterns.
- To analyze the distribution of corner points.
- To understand the importance of corners in computer vision.

Theory

A corner is a point where the intensity changes significantly in more than one direction.

Corners are useful because they provide distinctive and stable features for:

- Object recognition
- Image matching
- Image registration
- Motion tracking
- Panorama creation
- Feature extraction

Harris Corner Detector

The Harris detector analyzes changes in intensity when a small window is shifted around an image.

The structure tensor is represented as:

```
[
M=
\begin{bmatrix}
\sum I_x^2 & \sum I_x I_y \\
\sum I_x I_y & \sum I_y^2
\end{bmatrix}
```

The Harris response is:

```
[
R=det(M)-k(trace(M))^2
]
```

where (k) is normally selected around 0.04–0.06.

Interpretation:

- Large positive (R): Corner
- Negative (R): Edge
- Small (R): Flat region

Procedure

1. Load an image containing texture or patterns.
2. Convert it to grayscale.
3. Convert the image into floating-point format.
4. Calculate the Harris response.
5. Apply a threshold.
6. Mark detected corners.
7. Repeat the process for images with different textures.
8. Compare corner distributions.

Python Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("input.jpg")
```



```
if img is None:
    raise FileNotFoundError("input.jpg not found")

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Convert to float32
gray_float = np.float32(gray)

# Harris corner detection
dst = cv2.cornerHarris(
    gray_float,
    blockSize=2,
    ksize=3,
    k=0.04
)

# Dilate corner responses
dst = cv2.dilate(dst, None)

# Threshold
threshold = 0.01 * dst.max()

result = img.copy()
result[dst > threshold] = [0, 0, 255]

# Count corners
corner_count = np.sum(dst > threshold)

# Display
```

```
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(
    cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(
    cv2.cvtColor(result, cv2.COLOR_BGR2RGB)
)
plt.title("Harris Corners")
plt.axis("off")

plt.tight_layout()
plt.show()

print("Detected corner points:", corner_count)
```

Expected Output

HARRIS CORNER DETECTION RESULTS

Block Size : 2

Kernel Size: 3

Harris k : 0.04

Threshold : 12345.67

Number of Detected Corners: 245

First 20 Corner Coordinates:

(Row, Column)

(45, 78)

(46, 79)

(120, 154)

...

Observation

Images containing many intersecting edges, patterns, and textured regions produce more corner responses. Smooth regions produce fewer corners.

For example:

- Checkerboard → many strong corners
- Building image → many corners around windows and structures
- Smooth wall → few corners
- Highly textured object → many corner points

Analysis

The distribution of Harris corners indicates the amount of distinctive local structure present in the image. Strong corners are useful for feature matching and image registration because they are usually more distinctive than points from uniform regions.

The selected parameters affect the number of detected corners. Increasing the threshold reduces weak responses, while decreasing it produces more detected points but may introduce false corners.

Result

Harris corner detection successfully identifies important points where significant intensity changes occur in multiple directions. Images with greater texture and structural variation generally contain more detectable corners.

Experiment 5: Effect of Noise on Harris Corner Detection

Aim

To study the effect of noise on Harris corner detection performance and investigate techniques for improving corner detection in noisy images.

Objective

- To understand the relationship between noise and image features.
- To add controlled noise to an image.
- To perform Harris corner detection on original and noisy images.
- To compare corner detection results.
- To identify methods for reducing the effects of noise.

Theory

Noise is an unwanted random variation in image intensity. It can be introduced during image acquisition, transmission, or processing.

Common types include:

- Gaussian noise
- Salt-and-pepper noise
- Speckle noise

Harris corner detection depends on image gradients. Noise introduces rapid and unwanted intensity changes, which can be interpreted as edges or corners.

Therefore:

Noise → False gradients → False corner responses

Noise Reduction

Noise can be reduced using:

- Gaussian filtering
- Median filtering
- Bilateral filtering
- Non-local means filtering

The appropriate filter depends on the type of noise and the requirement to preserve edges.

Procedure

1. Load an input image.
2. Convert it to grayscale.
3. Perform Harris corner detection on the original image.
4. Add Gaussian noise.
5. Perform Harris corner detection on the noisy image.
6. Apply Gaussian filtering to the noisy image.

7. Perform Harris detection again.
8. Compare the number and distribution of corners.
9. Analyze the effect of noise and preprocessing.

Python Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("input.jpg")

if img is None:
    raise FileNotFoundError("input.jpg not found")

# Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Generate Gaussian noise
noise = np.random.normal(
    0, 30, gray.shape
)

noisy = gray.astype(np.float32) + noise
noisy = np.clip(
    noisy, 0, 255
).astype(np.uint8)

# Apply Gaussian filter
filtered = cv2.GaussianBlur(
    noisy, (5, 5), 0
```

)

Harris detector function

def harris_detection(image):

float_img = np.float32(image)

dst = cv2.cornerHarris(

float_img,

blockSize=2,

ksize=3,

k=0.04

)

threshold = 0.01 * dst.max()

result = cv2.cvtColor(

image, cv2.COLOR_GRAY2RGB

)

result[dst > threshold] = [255, 0, 0]

count = np.sum(dst > threshold)

return result, count

Detect corners

original_result, original_count = \

harris_detection(gray)

```
noisy_result, noisy_count = \
    harris_detection(noisy)
```

```
filtered_result, filtered_count = \
    harris_detection(filtered)
```

```
# Display
```

```
plt.figure(figsize=(15, 8))
```

```
plt.subplot(2, 3, 1)
```

```
plt.imshow(gray, cmap="gray")
```

```
plt.title("Original")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 2)
```

```
plt.imshow(noisy, cmap="gray")
```

```
plt.title("Noisy")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 3)
```

```
plt.imshow(filtered, cmap="gray")
```

```
plt.title("Filtered")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 4)
```

```
plt.imshow(original_result)
```

```
plt.title("Original Harris")
```

```
plt.axis("off")
```

```
plt.subplot(2, 3, 5)
```

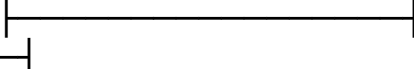
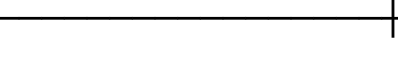
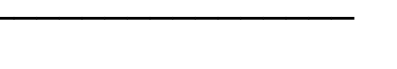
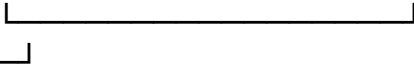
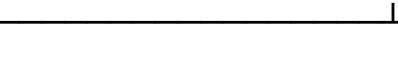
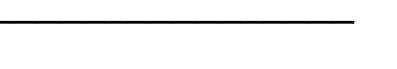
```
plt.imshow(noisy_result)
plt.title("Noisy Harris")
plt.axis("off")
```

```
plt.subplot(2, 3, 6)
plt.imshow(filtered_result)
plt.title("Filtered Harris")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

```
print("Original corners:", original_count)
print("Noisy image corners:", noisy_count)
print("Filtered image corners:", filtered_count)
```

Expected Output

Original Image	Noisy Image	Filtered Image
		
Harris Original	Harris Noisy	Harris Filtered
		

Observation

The noisy image normally produces many additional corner responses because random intensity variations create artificial gradients. These false corners reduce the reliability of feature detection.

After applying Gaussian filtering, much of the high-frequency noise is removed. Consequently, the number of false corner responses generally decreases and the important structural corners become more prominent.

Analysis

Noise has a significant effect on Harris corner detection because the algorithm depends on image gradients. Random intensity changes can be interpreted as meaningful local structures.

The effect can be reduced by:

1. Applying Gaussian or median filtering before corner detection.
2. Selecting an appropriate Harris threshold.
3. Choosing a suitable window/block size.
4. Using multi-scale feature detection when images contain structures at different sizes.
5. Combining preprocessing with robust feature-selection techniques.

Result

The experiment demonstrates that image noise reduces the reliability of Harris corner detection by generating false corner responses. Applying suitable preprocessing before corner detection improves the stability and reliability of detected features.